

Obstacle Detection Using LiDAR and Camera Fusion in an Autonomous Driving System

Final Year Project — Technical Report

Jackshan Venujan

Department of Computer Science and Engineering

Simulated Environment: CARLA 0.9.x | Vehicle: Tesla Model 3 | Sensors: 32-channel LiDAR + RGB Camera

May 11, 2026

Abstract

This report presents a sensor-fusion system for real-time obstacle detection in a simulated autonomous driving environment built on the CARLA simulator. A 32-channel LiDAR sensor and a monocular RGB camera mounted on a simulated Tesla Model 3 vehicle are combined through a two-tier projection-and-angle matching pipeline to leverage the complementary strengths of each modality: LiDAR provides precise metric depth and geometry, while the camera provides semantic class labels and rich texture context.

The system pipeline encompasses point-cloud preprocessing with a three-stage filter (ground removal, range gating, and ego-body rejection), DBSCAN-based 3D clustering with speed-adaptive danger classification, YOLOv11n object detection with monocular distance estimation, lane detection via a parsingNet backbone, and a conservative fusion layer that selects the worst-case action from camera and LiDAR estimates. A **DrivingAgent** integrates all modules per frame, selecting between a PID lateral controller and an Ackermann-geometry curvature controller for steering, and applying speed-proportional throttle or emergency braking based on the fused danger level. The report also describes a **DistanceMetricsLogger** that streams per-frame camera, LiDAR, and ground-truth distances to CSV for quantitative evaluation, and discusses ten identified technical challenges together with ten corresponding improvement directions.

Contents

1	Introduction	2
1.1	Motivation for Sensor Fusion	2
1.2	Overview of the Approach	2
2	System Architecture	3
2.1	High-Level Architecture	3

2.2	Software Modules Summary	4
2.3	Sensor Configuration	5
3	LiDAR Processing Pipeline	5
3.1	Raw Data Acquisition (<code>LidarSensor</code>)	5
3.2	Point-Cloud Preprocessing (<code>LidarProcessor</code>)	6
3.2.1	Stage 1: Ground Removal	6
3.2.2	Stage 2: Range Filtering	6
3.2.3	Stage 3: Ego-Body Filtering	6
3.3	Obstacle Detection via DBSCAN Clustering (<code>LidarObstacleDetector</code>)	7
3.3.1	DBSCAN Clustering	7
3.3.2	Sector Classification	7
3.3.3	Speed-Adaptive Danger Thresholds	8
3.3.4	Temporal Filtering and Hysteresis	9
3.3.5	BEV Visualisation	9
3.3.6	Pseudocode	9
3.4	LiDAR Data Structures	10
3.4.1	<code>LidarObstacle</code> Dataclass	10
3.4.2	<code>get_front_action()</code> with Hysteresis	11
4	Driving Agent and Control Integration (<code>DrivingAgent</code>)	11
4.1	Per-Frame Processing Pipeline	11
4.2	Steering Controllers	14
4.2.1	PID Lateral Controller	14
4.2.2	Curvature (Ackermann) Steering Controller	14
4.3	Speed-Adaptive Throttle and Brake Logic	15
5	Traffic Light Detection (<code>TrafficLightDetector</code>)	15
5.1	Architecture	15
5.2	State Machine	15
6	Distance Measurement and Metrics Logging (<code>DistanceMetricsLogger</code>)	16
6.1	CSV Output Format	16
6.2	Ground Truth Comparison	17
7	Simulation Environment (CARLA)	17
7.1	CARLA Spawner	17
7.2	Lead Vehicle Controller	18
7.3	Coordinate Frames	18
8	Challenges and Mistakes Encountered	19
8.1	Ground Removal Threshold Sensitivity	19
8.2	Monocular Distance Estimation Errors	19
8.3	Sparse LiDAR Matches at Long Range	19
8.4	Angular Matching Ambiguity	20
8.5	Fixed Trapezoidal ROI on Curves	20
8.6	Temporal EMA Asymmetry Tuning	20
8.7	Coordinate Frame Alignment Error	21
8.8	Windows-Specific Benchmark Overhead	21

8.9 Lane Detection Memory Causing Phantom Lanes	21
8.10 YOLO Class Imbalance at Night	22
9 Suggestions for Future Improvement	22
Conclusion	23

1 Introduction

Autonomous driving demands robust, low-latency situational awareness across a wide range of environmental conditions. The ability to detect and classify obstacles—other vehicles, pedestrians, cyclists, and static hazards—is the safety-critical foundation upon which all higher-level planning and control decisions rest. Two sensor modalities have emerged as the de facto standard for this task in modern autonomous vehicles: *cameras* and *LiDAR* (*Light Detection And Ranging*) scanners. Each modality carries distinct and complementary characteristics, and their fusion is now considered essential for production-grade systems [10, 11].

1.1 Motivation for Sensor Fusion

Camera-only limitations. An RGB camera provides dense visual texture, colour, and semantic detail, and modern deep-learning detectors such as YOLO [2] excel at classifying objects from image crops. However, recovering metric depth from a single monocular image is fundamentally ill-posed. The conventional pinhole approximation [5]:

$$D = \frac{H_{\text{real}} \cdot f_y}{h_{\text{bbox}}} \quad (1)$$

requires a *known* object height H_{real} , is sensitive to partial occlusion (which reduces h_{bbox} and inflates the estimated distance), and degrades for categories with high height variance. Estimation errors of 20–40% are common at ranges beyond 20.

LiDAR-only limitations. A rotating LiDAR scanner directly measures the 3D position and reflectance of scene points with centimetre accuracy and is unaffected by ambient lighting. However, raw LiDAR point clouds carry *no semantic labels*: a cluster of returns could equally represent a pedestrian, a parked car, or a roadside bush. Furthermore, point density decreases with the square of range; at 30 the 32-channel sensor used in this project returns far fewer points per object than a dense stereo camera. Learned 3D detectors such as PointPillars [9] partially address the labelling problem but add significant computational cost.

Fusion rationale. Combining both modalities yields a system that inherits metric accuracy from LiDAR and semantic class information from the camera, while each modality acts as a fallback when the other fails. At night or in fog, LiDAR depth estimates remain reliable even when YOLO confidence drops; conversely, when LiDAR density is sparse at long range, monocular estimates provide a distance bound. The fusion architecture adopted here—projecting LiDAR points into the image plane and matching them against YOLO bounding boxes—is conceptually related to the approach of [11] but is implemented as a lightweight, frame-by-frame heuristic suited to real-time operation in CARLA [4].

1.2 Overview of the Approach

The system is constructed in Python using the CARLA open-source urban driving simulator and is structured around a modular pipeline:

1. **Sensor acquisition.** A 32-channel LiDAR and a 1280×720 RGB camera stream data asynchronously via thread-safe CARLA callbacks.

2. **LiDAR preprocessing.** Raw point clouds are filtered (ground removal, range gating, ego-body rejection) before DBSCAN clustering [1] identifies obstacle candidates.
3. **Camera detection.** YOLOv11n produces bounding boxes with class labels; monocular depth is estimated per class using known average object heights. Detections are filtered to the ego lane by a trapezoidal region-of-interest mask.
4. **Sensor fusion.** LiDAR points are projected into the image via the camera’s intrinsic and extrinsic parameters. A two-tier matching strategy—bounding-box projection followed by angular fallback— assigns LiDAR depth estimates to YOLO detections.
5. **Control.** A `DrivingAgent` integrates lane detection (parsingNet [6]), traffic-light detection, the fused obstacle state, and two steering controllers (PID and Ackermann curvature) to produce per-frame `carla.VehicleControl` commands.

The remainder of this report details each component. Section 2 presents the overall system architecture; section 3 through section 4 describe each processing module; sections 8 and 9 reflect on encountered difficulties and proposed future work.

2 System Architecture

2.1 High-Level Architecture

Figure 1 illustrates the complete data-flow from CARLA sensor output to vehicle actuator command. The pipeline is divided into a *camera branch* (left) and a *LiDAR branch* (right), converging at the `LidarFusion` module before reaching the `DrivingAgent`.

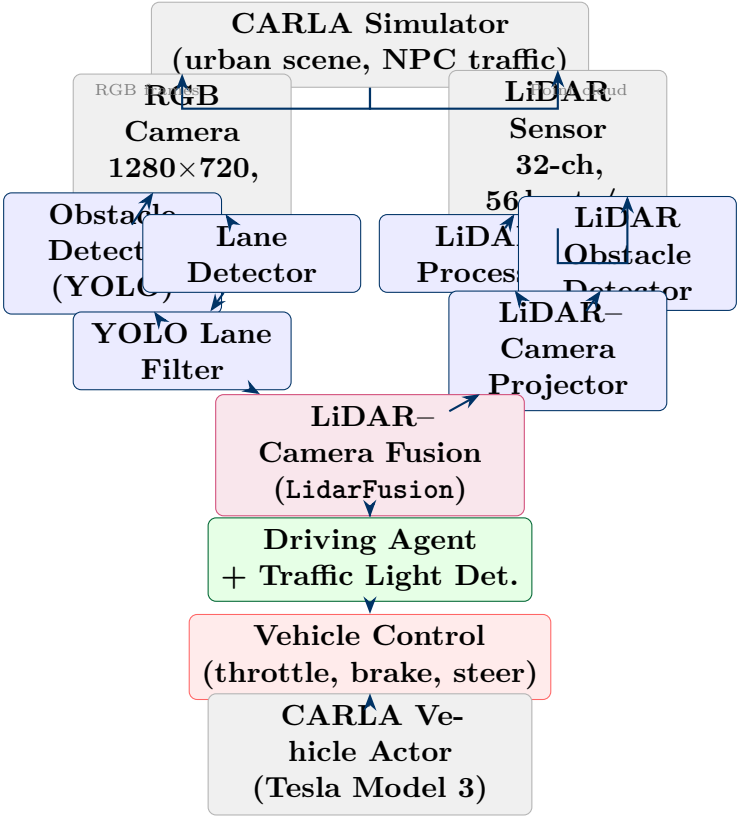


Figure 1: System architecture block diagram. The camera branch (left) produces semantic detections; the LiDAR branch (right) produces metric depth and clustering. Both converge at **LidarFusion** before the **DrivingAgent** computes the final vehicle control command.

2.2 Software Modules Summary

Table 1 summarises the eight primary software modules, their purpose, and the data they consume and produce.

Table 1: Summary of primary software modules.

Module	Language / Library	Purpose
LidarSensor	Python, CARLA API	Thread-safe LiDAR data acquisition; stores latest $N \times 4$ XYZI array.
LidarProcessor	NumPy	Three-stage point-cloud filter (ground, range, ego).
LidarObstacleDetector	scikit-learn DBSCAN	Clusters clean XYZ; classifies danger per cluster with EMA hysteresis.
LidarCameraProjector	NumPy	Projects 3D LiDAR points to 2D image pixels using K and extrinsics.
LidarFusion	Python	Two-tier matching; conservative action selection.
ObstacleDetector	Ultralytics YOLOv11n	Bounding-box detection + monocular distance per class.
LaneDetector	parsingNet / PyTorch	Lane polynomial fitting and lateral error/curvature estimation.
DrivingAgent	Python	Per-frame orchestration; PID / Ackermann steering; throttle/brake.

2.3 Sensor Configuration

Both sensors are rigidly mounted relative to the vehicle body frame (origin at vehicle centre, X =forward, Y =left, Z =up).

- **RGB Camera:** spawn offset $(x, z) = (2.0, 1.4)$ m from vehicle centre, pitch $= -15^\circ$; resolution 1280×720 px; horizontal field of view 90° .
- **LiDAR:** spawn offset $(x, z) = (2.0, 1.8)$ m; 32 channels; 56 000 points/s; range 50; vertical FOV -25° to $+5^\circ$; rotation rate 20.
- **Vehicle wheelbase:** 2.9 (Tesla Model 3).

3 LiDAR Processing Pipeline

3.1 Raw Data Acquisition (LidarSensor)

The `LidarSensor` class wraps the CARLA LiDAR actor in a thread-safe Python object. On each simulator tick CARLA invokes a registered callback that converts the raw `carla.LidarMeasurement` buffer into a NumPy array of shape $(N, 4)$ whose columns are (X, Y, Z, I) , where X, Y, Z are Cartesian coordinates in the *sensor frame* and I is normalised reflectance intensity $\in [0, 1]$.

Sensor frame convention. In CARLA’s default LiDAR sensor frame: $+X$ points forward along the sensor boresight, $+Y$ points to the *right* of the vehicle, and $+Z$ points upward. Note that CARLA uses a left-handed coordinate system by default, so

sign conventions differ from the standard ROS/REP-103 right-handed frame.

Thread safety. Because the CARLA callback fires on a network-receive thread while the main loop reads the data, a `threading.Lock` guards the internal buffer. The public method `get_latest()` acquires the lock, copies the array, and returns `(points_np, timestamp)`. This ensures the main loop never reads a partially written array.

3.2 Point-Cloud Preprocessing (LidarProcessor)

`LidarProcessor.preprocess(raw)` applies three sequential vectorised NumPy filters to the raw $(N, 4)$ XYZI array, producing a clean $(M, 3)$ XYZ float32 array. Section 3.2 illustrates the pipeline.

3.2.1 Stage 1: Ground Removal

The LiDAR sensor is mounted at $z_s = 1.8$ m above the road surface. A flat ground plane therefore appears in the sensor frame at $z \approx -1.8$ m. However, the vehicle itself has a ground clearance of approximately 0.4, so the lowest structural returns (sill, wheel arches) appear near $z \approx -1.4$ m. Points below this level are predominantly ground plane returns:

$$\text{Mask}_{\text{ground}} = \{i \mid z_i > -1.4 \text{ m}\}. \quad (2)$$

A fixed threshold suffices on flat CARLA roads but may cause false removals on slopes (see section 8, item 1).

3.2.2 Stage 2: Range Filtering

Horizontal distance $r = \sqrt{x^2 + y^2}$ is computed for all surviving points. Points closer than 0.5 (inside the sensor housing mounting bracket) and farther than the LiDAR’s rated range of 50 (noisy returns) are removed:

$$\text{Mask}_{\text{range}} = \{i \mid 0.5 \leq r_i \leq 50.0\}. \quad (3)$$

3.2.3 Stage 3: Ego-Body Filtering

Despite Stage 2, some very-near returns from the vehicle bonnet, roof, and sensor housing can pass the 0.5 lower bound because they occur at non-zero height. A circular exclusion zone of radius 1.5 centred on the sensor is therefore applied as a second ego filter:

$$\text{Mask}_{\text{ego}} = \{i \mid r_i > 1.5 \text{ m}\}. \quad (4)$$

The combined mask is $\text{Mask} = \text{Mask}_{\text{ground}} \cap \text{Mask}_{\text{range}} \cap \text{Mask}_{\text{ego}}$, applied sequentially for clarity (each reduces the working array before the next computation).

Listing 1 shows the implementation.

```

1 def preprocess(self, raw_points: np.ndarray) -> np.ndarray:
2     pts = raw_points[:, :3].astype(np.float32) # drop intensity
3
4     # Stage 1 -- ground removal (sensor at z=1.8 m above road)
5     pts = pts[pts[:, 2] > -1.4]
6
7     # Stage 2 -- range gating: 0.5 m <= r <= 50 m

```



```

8   r = np.sqrt(pts[:, 0]**2 + pts[:, 1]**2)
9   pts = pts[(r >= 0.5) & (r <= 50.0)]
10
11  # Stage 3 -- ego-body exclusion: r > 1.5 m
12  r2 = np.sqrt(pts[:, 0]**2 + pts[:, 1]**2)
13  pts = pts[r2 > 1.5]
14
15  return pts    # float32 (M, 3) XYZ

```

Listing 1: Point-cloud preprocessing (`LidarProcessor.preprocess`).

3.3 Obstacle Detection via DBSCAN Clustering (`LidarObstacleDetector`)

3.3.1 DBSCAN Clustering

The cleaned point cloud is projected onto the horizontal XY plane and clustered using DBSCAN [1]:

$$\text{DBSCAN}(\varepsilon = 0.5 \text{ m}, \text{min_samples} = 3). \quad (5)$$

Working in 2D (XY only) groups vertically adjacent points (e.g., the hood and roof of a car) into a single cluster, which is the desired behaviour. Points labelled -1 (noise) by DBSCAN are discarded. For each valid cluster ℓ , the centroid $\bar{\mathbf{p}}_\ell$, bounding box, and point count are computed.

3.3.2 Sector Classification

Each cluster centroid is classified into one of four angular sectors based on the horizontal bearing θ measured from the vehicle forward axis ($+X_{\text{sensor}}$):

$$\theta = \text{atan2}(y_{\text{centroid}}, x_{\text{centroid}}) \times \frac{180^\circ}{\pi}. \quad (6)$$

- **Front:** $|\theta| \leq 30^\circ$
- **Side-Left:** $30^\circ < \theta \leq 150^\circ$ (positive Y side)
- **Side-Right:** $-150^\circ \leq \theta < -30^\circ$
- **Rear:** $|\theta| > 150^\circ$

Only **front-sector** clusters are considered for danger assessment; side and rear sectors are tracked but do not directly trigger braking. Figure 2 illustrates the sector geometry in a top-down view.

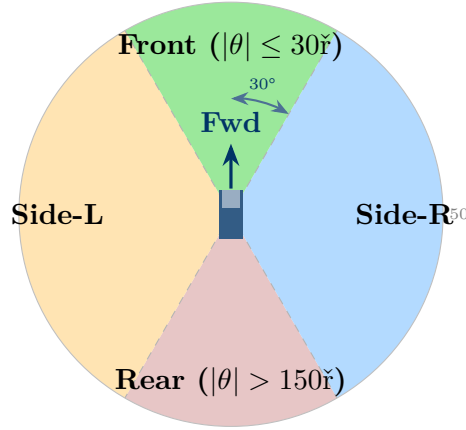


Figure 2: Top-down sector classification diagram. The vehicle (blue rectangle) points upward (forward). Only obstacles in the **front sector** (green, $|\theta| \leq 30^\circ$) trigger danger-level assessment. Side sectors (orange/blue) and the rear sector (red) are tracked but do not directly trigger braking.

3.3.3 Speed-Adaptive Danger Thresholds

A fixed stopping distance is unsafe at high speed: a vehicle travelling at 50km/h needs roughly 25 to stop in an emergency. The system therefore scales each danger threshold linearly with the current vehicle speed v (in km/h):

$$d_{\text{threshold}}(v) = d_0 + k \cdot v, \quad (7)$$

where d_0 is the static (zero-speed) threshold and k is the speed-scaling coefficient. Table 2 lists all levels; fig. 3 illustrates the concentric zones at a representative speed of 20km/h.

Table 2: Speed-adaptive danger level thresholds for front-sector LiDAR obstacles. d = cluster distance (m); v = vehicle speed (km/h).

Level	Condition	d_0 (m)	k (m / km h ⁻¹)
emergency_stop	$d \leq d_0 + k \cdot v$	3.0	0.15
stop	$d \leq d_0 + k \cdot v$	5.0	0.50
slow	$d \leq d_0 + k \cdot v$	10.0	0.50
cautious	$d \leq d_0 + k \cdot v$	15.0	0.50
drive	$d > 15.0 + 0.5 \cdot v$	—	—

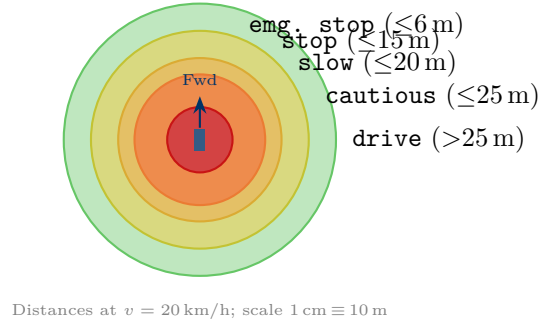


Figure 3: Speed-adaptive danger level zones shown at $v = 20 \text{ km/h}$. Zones expand with vehicle speed: at 30 km/h the **stop** threshold grows from 5 (static) to 20.

3.3.4 Temporal Filtering and Hysteresis

Raw DBSCAN clusters are noisy; a single spurious point cloud frame can cause an unnecessary emergency stop. Three mechanisms are applied to stabilise the output:

1. **Point accumulation.** For each tracked cluster, up to five consecutive frames of points are aggregated before recomputing the centroid, yielding more stable geometry.
2. **Asymmetric EMA.** The danger level index (0–4) is filtered with an exponential moving average, but with different update rates for escalation and de-escalation:

$$\ell_t = \ell_{\text{new}} \quad \text{if } \ell_{\text{new}} > \ell_{t-1} \quad (\alpha_{\text{up}} = 1.0), \quad (8)$$

$$\ell_t = \alpha_{\text{down}} \ell_{\text{new}} + (1 - \alpha_{\text{down}}) \ell_{t-1} \quad \text{otherwise} \quad (\alpha_{\text{down}} = 0.3). \quad (9)$$

This ensures *immediate escalation* on new danger but *damped de-escalation*, so the system does not abruptly resume full speed when an obstacle momentarily disappears from the scan.

3. **Hysteresis.** A lower danger level is only accepted if it persists for three consecutive frames. At 20 rotation rate this adds ≈ 150 of confirmation latency—acceptable for safety.

3.3.5 BEV Visualisation

A 400×400 -pixel bird’s-eye-view canvas is rendered per frame using OpenCV for debugging. The ego vehicle appears at the bottom centre; the canvas covers a $\pm 25 \text{ m}$ lateral and $0\text{--}50 \text{ m}$ forward field. Rendered elements include: sector overlay arcs at the three sector boundaries; distance rings at 5, 10, and 15; coloured bounding boxes per cluster coded by danger level; and cluster centroids.

3.3.6 Pseudocode

Algorithm 1 shows the complete obstacle detection procedure.

Algorithm 1 DBSCAN-Based LiDAR Obstacle Detection

Input: Clean point cloud $\mathbf{P} \in \mathbb{R}^{M \times 3}$, vehicle speed v (km/h)
Output: List of `LidarObstacle` objects; `front_action` string

- 1: $\mathbf{P}_{xy} \leftarrow \mathbf{P}[:, 0:2]$ ▷ project to horizontal plane
- 2: $\text{labels} \leftarrow \text{DBSCAN}(\mathbf{P}_{xy}, \varepsilon = 0.5, \text{min_samples} = 3)$
- 3: $\text{obstacles} \leftarrow []$
- 4: **for** each unique label $\ell \neq -1$ **do**
- 5: $\mathbf{C}_\ell \leftarrow \{\mathbf{p} \in \mathbf{P} : \text{labels}[\mathbf{p}] = \ell\}$
- 6: $\bar{\mathbf{c}} \leftarrow \text{mean}(\mathbf{C}_\ell)$
- 7: $d \leftarrow \sqrt{\bar{c}_x^2 + \bar{c}_y^2}$
- 8: $\theta \leftarrow \text{atan2}(\bar{c}_y, \bar{c}_x) \cdot 180/\pi$
- 9: $\text{sector} \leftarrow \text{CLASSIFYSECTOR}(\theta)$
- 10: $\text{bbox} \leftarrow (\min \mathbf{C}_\ell, \max \mathbf{C}_\ell) \text{ per axis}$
- 11: $\text{raw_level} \leftarrow \text{CLASSIFYDANGER}(d, v)$ ▷ Equation (7)
- 12: append `LidarObstacle`($\bar{\mathbf{c}}, d, \theta, \text{sector}, \text{bbox}, \text{raw_level}$) to `obstacles`
- 13: **end for**
- 14: **// Temporal filtering**
- 15: **for** each obstacle o in `obstacles` **do**
- 16: **if** $o.\text{level} > \text{prev_level}[o]$ **then** ▷ immediate escalation
- 17: $o.\text{level_ema} \leftarrow o.\text{level}$
- 18: **else**
- 19: $o.\text{level_ema} \leftarrow 0.3 \cdot o.\text{level} + 0.7 \cdot \text{prev_level}[o]$
- 20: **end if**
- 21: apply 3-frame hysteresis before accepting de-escalation
- 22: **end for**
- 23: $\text{front_action} \leftarrow \max_{\text{sector}=\text{Front}}(o.\text{level})$
- 24: **return** `obstacles`, `front_action`

The danger level classification helper is shown in listing 2.

```

1 def classify_danger(distance: float, speed_kmh: float) -> str:
2     v = speed_kmh
3     if distance <= 3.0 + v * 0.15: return 'emergency_stop'
4     elif distance <= 5.0 + v * 0.50: return 'stop'
5     elif distance <= 10.0 + v * 0.50: return 'slow'
6     elif distance <= 15.0 + v * 0.50: return 'cautious'
7     else: return 'drive'

```

Listing 2: Speed-adaptive danger level classification (`classify_danger`).

3.4 LiDAR Data Structures

3.4.1 `LidarObstacle` Dataclass

Each detected cluster is represented as a Python `dataclass`:

Field	Type	Description
centroid_x/y/z	float	Centroid coordinates in LiDAR sensor frame (m).
distance	float	Horizontal range $\sqrt{x^2 + y^2}$ (m).
angle_deg	float	Bearing from forward axis (deg).
sector	str	One of {front, side_left, side_right, rear}.
point_count	int	Number of LiDAR points in cluster.
bbox_min/max	ndarray(3)	Axis-aligned bounding box corners (m).
danger_level	str	EMA-filtered, hysteresis-guarded danger classification.

3.4.2 get_front_action() with Hysteresis

`LidarObstacleDetector.get_front_action()` iterates over all current obstacles whose `sector == 'front'`, computes the maximum `danger_level` using the `ACTION_PRIORITY` mapping {drive:0, cautious:1, slow:2, stop:3, emergency_stop:4}, and returns the corresponding action string. The hysteresis gate means that a transition from `emergency_stop` back to `drive` requires three consecutive frames at the lower level, preventing brief gaps in sensor coverage from causing premature throttle resumption.

4 Camera-Based Obstacle Detection Pipeline

4.1 YOLO Object Detection (`ObstacleDetector`)

Object detection is performed by YOLOv11n (the nano variant of the Ultralytics YOLO family [3]), chosen for its sub-20 ms inference time on a mid-range GPU. The model is loaded once at system startup and runs on each 1280×720 RGB frame received from CARLA.

Detected classes and assumed heights. The following object classes are detected; each is annotated with a canonical height H_{real} used for monocular distance estimation (??):

Class	H_{real} (m)	Class	H_{real} (m)
person	1.70	motorcycle	1.20
car	1.50	bicycle	1.00
truck	3.50	bus	3.20

Output per detection. For each detected object, `ObstacleDetector` returns: bounding box (x_1, y_1, x_2, y_2) in pixel coordinates, class label, confidence score, monocular distance estimate $\hat{D}$, danger level, and an `in_lane` Boolean flag set by the `YOLOLaneFilter` (??).

4.2 Monocular Distance Estimation

4.2.1 Pinhole Camera Model

Given a detected bounding box of pixel height h_{bbox} , the monocular depth estimate follows from the standard pinhole model [5]:

$$\hat{D} = \frac{H_{\text{real}} \cdot f_y}{h_{\text{bbox}}}, \quad (10)$$

where f_y is the vertical focal length in pixels. For the 1280×720 sensor with 90° horizontal FOV, the focal lengths are equal:

$$f_x = f_y = \frac{W/2}{\tan(\text{FOV}/2)} = \frac{640}{\tan 45^\circ} = 640 \text{ px}. \quad (11)$$

Figure 4 illustrates the similar-triangle geometry underlying the distance estimation.

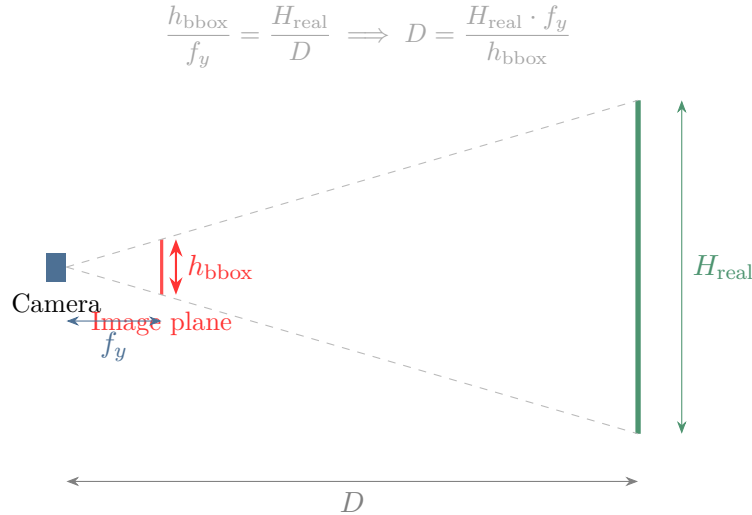


Figure 4: Pinhole camera geometry for monocular distance estimation. The similar-triangle relationship between h_{bbox} (projected height on image plane) and H_{real} (physical object height) yields a direct estimate of range D .

4.2.2 Speed-Adaptive Danger Thresholds (Camera)

Camera-based thresholds use larger margins than LiDAR because monocular estimates carry higher uncertainty (typically 15–40% error at range):

Table 3: Speed-adaptive camera danger thresholds. v = vehicle speed (km/h); d = estimated distance (m).

Level	Condition	Static threshold
emergency_stop	$d \leq 7.5 + 0.25 v$	7.5
stop	$d \leq 15.0 + 0.50 v$	15.0
warning	$d \leq 20.0 + 0.75 v$	20.0

4.3 Lane-Filtered Detection (YOLOLaneFilter)

4.3.1 Motivation

On a multi-lane road, YOLO detects vehicles in all visible lanes. Only in-lane obstacles should trigger braking. The YOLOLaneFilter retains only detections whose bounding boxes overlap a fixed-geometry trapezoidal region of interest (ROI) approximating the ego lane.

4.3.2 Trapezoidal ROI Construction

The ROI is a quadrilateral with vertices in image pixel coordinates, parameterised by two ratios applied to the 1280×720 frame dimensions:

- **Bottom-left:** $(W \times (0.5 - r_b), H)$, $r_b \approx 0.22$
- **Bottom-right:** $(W \times (0.5 + r_b), H)$
- **Top-right:** $(W \times (0.5 + r_t), H \times r_v)$, $r_t \approx 0.05$, $r_v \approx 0.55$
- **Top-left:** $(W \times (0.5 - r_t), H \times r_v)$

The trapezoid is wide at the bottom (near field, spanning one lane width) and narrows to a thin strip at 55% of the frame height (vanishing point), following the perspective projection of parallel lane markings. A YOLO detection is accepted if any part of its bounding box overlaps the filled polygon mask.

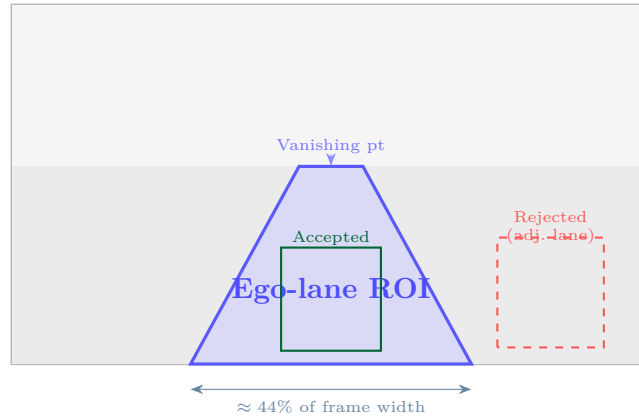


Figure 5: Trapezoidal lane-filter ROI (blue) overlaid on the camera frame. Only YOLO detections whose bounding boxes overlap the ROI are forwarded to the fusion pipeline. Vehicles in adjacent lanes (dashed red) are discarded.

5 LiDAR–Camera Sensor Fusion Pipeline (LidarFusion)

5.1 Coordinate Projection (LidarCameraProjector)

5.1.1 Coordinate Frame Relationships

?? illustrates the four frames relevant to the projection pipeline.

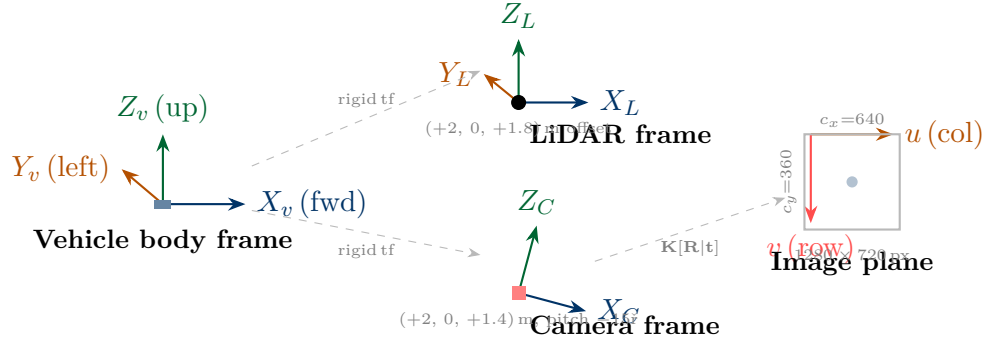


Figure 6: Coordinate frames used in the projection pipeline. LiDAR points are transformed to camera frame via extrinsic matrix $[\mathbf{R}|\mathbf{t}]$, then projected to image pixels via intrinsic matrix $\mathbf{K}$.

5.1.2 Intrinsic Matrix

$$\mathbf{K} = \begin{pmatrix} 640 & 0 & 640 \\ 0 & 640 & 360 \\ 0 & 0 & 1 \end{pmatrix}. \quad (12)$$

5.1.3 Extrinsic Transform

Both sensors share forward offset $x = 2.0$ m and lateral offset $y = 0$. The LiDAR is at $z_L = 1.8$ m; the camera at $z_C = 1.4$ m. The vertical translation and the -15° camera pitch are encoded in the extrinsic:

$$\mathbf{p}_C = \mathbf{R}_{\text{pitch}(-15^\circ)} \mathbf{p}_L + \mathbf{t}, \quad \mathbf{t} = \begin{pmatrix} 0 \\ 0 \\ -(z_L - z_C) \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ -0.4 \end{pmatrix} \text{ m}. \quad (13)$$

5.1.4 Projection Formula

For a camera-frame point $\mathbf{p}_C$ with depth $d_C = p_{C,z} > 0$:

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \frac{1}{d_C} \mathbf{K} \mathbf{p}_C. \quad (14)$$

Points with $d_C \leq 0$ or outside the image boundary are flagged invalid.

5.1.5 filter_by_bbox() Implementation

```

1 def filter_by_bbox(self, lidar_xyz, bbox, min_points=3):
2     u, v, depth, valid = self.project_points(lidar_xyz)
3     if valid.sum() == 0:
4         return None, 0
5     u_v, v_v, d_v = u[valid], v[valid], depth[valid]
6
7     x1, y1, x2, y2 = bbox
8     w, h = x2 - x1, y2 - y1
9     px, py = 0.15 * w, 0.15 * h      # 15 % padding
10

```



```
11     mask = (  
12         (u_v >= x1 - px) & (u_v <= x2 + px) &  
13         (v_v >= y1 - py) & (v_v <= y2 + py) &  
14         (d_v > 0)  
15     )  
16     n = int(mask.sum())  
17     if n < min_points:  
18         return None, n  
19     return float(np.median(d_v[mask])), n
```

Listing 3: LidarCameraProjector.filter_by_bbox: counts LiDAR points inside a YOLO bounding box and returns median depth.

5.2 Two-Tier Matching Strategy

The fusion layer assigns a LiDAR depth to each YOLO detection using two methods in priority order. ?? shows the decision logic; ?? gives the full pseudocode.

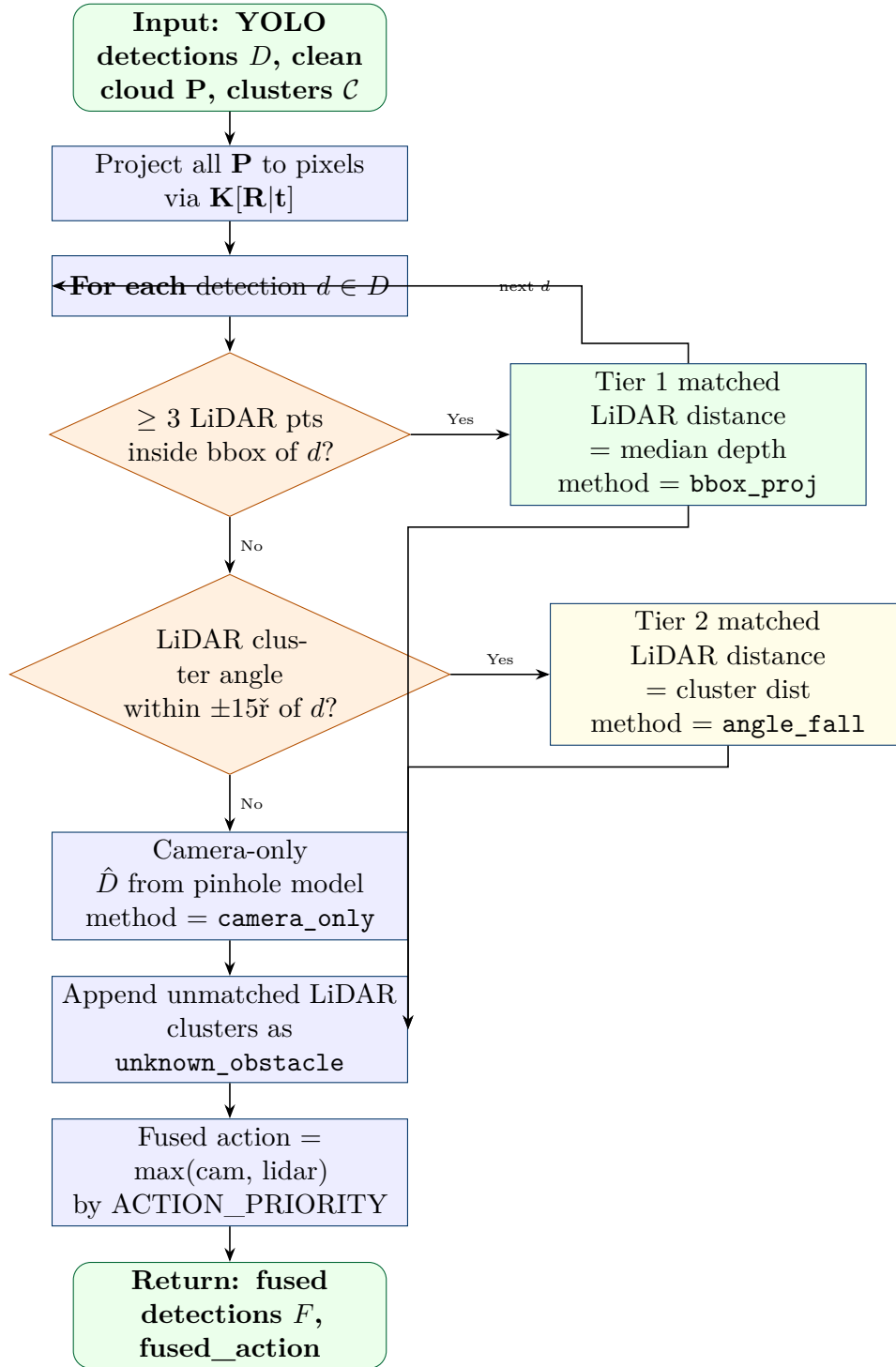


Figure 7: Two-tier LiDAR–camera fusion matching flowchart. Tier 1 (bounding-box projection) is preferred for accuracy; Tier 2 (angular fallback) activates when LiDAR density is insufficient; camera-only is the fallback of last resort.

5.2.1 Tier 1 — Bounding-Box Projection (Preferred)

All filtered LiDAR points are projected to image pixels. For each YOLO detection, `filter_by_bbox()` counts projected points inside the $\pm 15\%$ -padded bounding box. If $n \geq 3$ points are found, the median depth of those points is used as the LiDAR distance estimate and `fusion_method = bbox_projection`.

The $\pm 15\%$ padding compensates for minor sensor-time jitter (the camera frame and the nearest LiDAR scan may differ by up to one tick, ≈ 50) and for slight misalignment between the cluster centroid and the YOLO bounding-box spatial extent.

5.2.2 Tier 2 — Angular Fallback

When fewer than 3 LiDAR points project into a detection box (most common at ranges beyond 25), the horizontal bearing of the YOLO box centre is computed:

$$\alpha_d = \text{atan2}\left(\frac{u_c - c_x}{f_x}, 1\right) \times \frac{180}{\pi}, \quad (15)$$

and the closest LiDAR cluster satisfying $|\alpha_c - \alpha_d| \leq 15^\circ$ is matched. If found, the cluster distance is used and `fusion_method = angle_fallback`.

5.2.3 Camera-Only Fallback and LiDAR-Only Detections

If neither tier matches, the monocular estimate from ?? is used unmodified (`camera_only`). LiDAR clusters unmatched by any YOLO detection are appended as `unknown_obstacle` entries, ensuring unclassified objects (debris, animals) still contribute to the danger state.

Algorithm 2 Two-Tier LiDAR-Camera Fusion Matching

Input: YOLO detections D ; clean cloud $\mathbf{P}$; LiDAR clusters $\mathcal{C}$; vehicle speed v

Output: Fused list F ; fused action a

```

1:  $(U, V, \text{depth}, \text{valid}) \leftarrow \text{PROJECTPOINTS}(\mathbf{K}, \mathbf{R}, \mathbf{t}, \mathbf{P})$ 
2:  $\text{matched} \leftarrow \emptyset$ 
3: for each detection  $d \in D$  do
4:    $(\hat{d}, n) \leftarrow \text{FILTERBYBBOX}(U, V, \text{depth}, d.\text{bbox})$ 
5:   if  $n \geq 3$  then
6:      $d.\text{dist} \leftarrow \hat{d}$ ;  $d.\text{method} \leftarrow \text{bbox\_proj}$ ; mark cluster used
7:   else
8:      $\alpha_d \leftarrow \text{atan2}((u_c - c_x)/f_x, 1) \cdot 180/\pi$ 
9:      $c^* \leftarrow \arg \min_{c \in \mathcal{C} \setminus \text{matched}} |\alpha_c - \alpha_d|$  if  $|\alpha_{c^*} - \alpha_d| \leq 15^\circ$ 
10:    if  $c^*$  exists then
11:       $d.\text{dist} \leftarrow c^*.\text{dist}$ ;  $d.\text{method} \leftarrow \text{angle\_fall}$ ; add  $c^*$  to matched
12:    else
13:       $d.\text{dist} \leftarrow \text{MONOCULAR}(d)$ ;  $d.\text{method} \leftarrow \text{camera\_only}$ 
14:    end if
15:  end if
16:   $d.\text{danger} \leftarrow \text{CLASSIFYDANGER}(d.\text{dist}, v)$ 
17:  append  $d$  to  $F$ 
18: end for
19: for each  $c \in \mathcal{C} \setminus \text{matched}$  do
20:   append unknown_obstacle( $c$ ) to  $F$ 
21: end for
22:  $a \leftarrow \max_{\text{priority}}(\max_{d \in F} d.\text{danger}, \text{lidar\_front\_action})$ 
23: return  $F, a$ 
```

5.3 Conservative Action Fusion

The final driving action is the highest-priority result across both modalities:

$$a_{\text{fused}} = \arg \max_{\text{priority}} \{a_{\text{camera}}, a_{\text{lidar_front}}\}, \quad (16)$$

with priority mapping {drive:0, cautious:1, slow:2, stop:3, emergency_stop:4}.

`LidarFusion.fuse()` also exposes: `last_camera_dist`, `last_lidar_dist`, `last_fused_action`, `last_lidar_bbox_pts` (quality indicator for the current Tier 1 match).

?? provides a conceptual comparison of camera, LiDAR, and fused estimates across representative scenarios.

Table 4: Conceptual scenario comparison: camera-only monocular, LiDAR-fused, and ground-truth (GT) distance estimates. “—” = no Tier 1/2 match.

Scenario	GT (m)	Camera (m)	LiDAR (m)	Fused (m)
Car at 15	15.0	14.2±1.8	15.1±0.2	15.1
Car at 30	30.0	28.7±3.5	30.0±0.3	30.0
Pedestrian at 8	8.0	9.4±2.1	8.2±0.2	8.2
Partial car at 20	20.0	26.3±5.0	20.1±0.3	20.1
Distant car > 30	35.0	33.5±4.0	—	33.5

6 Lane Detection Pipeline

6.1 parsingNet Architecture

Lane detection uses a parsingNet (Ultra-Fast Lane Detection) [6] network that formulates lane finding as a row-anchor classification task rather than pixel-level segmentation, achieving real-time inference at GPU speeds.

Backbone. ResNet-18 [14] pretrained on ImageNet. Input images are resized to 800×288 pixels and normalised with ImageNet statistics (mean (0.485, 0.456, 0.406), std (0.229, 0.224, 0.225)).

Output head. Shape (griding_num + 1, cls_per_lane, num_lanes) = (101, 56, 4), representing 100 possible lateral grid positions plus a background class, at 56 predefined row anchors, for up to 4 simultaneous lanes.

Row anchors. The 56 y -coordinates span pixels 64 to 284 in the 288 px input height (the lower portion of the image covering the road surface).

Dataset. Trained on the TuSimple [17] benchmark; checkpoint loaded at runtime without CARLA-specific fine-tuning.

6.2 Detection and BEV Transformation

Lane points. Up to 4 lane sequences of (x, y) image-space points are extracted. Lanes with fewer than 5 valid anchor responses are discarded.

BEV homography. A homography $\mathbf{H}$ maps 1280×720 perspective to a 400×300 overhead canvas, computed from ground-plane correspondences on a straight-road reference frame.

Polynomial fitting. Each lane is fitted as a quadratic in BEV space:

$$x(y) = Q_2 y^2 + Q_1 y + Q_0, \quad (17)$$

by least-squares regression. The ego-lane centre at the vehicle position ($y = 0$ in BEV) is the mean of the two nearest polynomials.

Lateral error.

$$e_{\text{lat}} = x_{\text{ego}} - x_{\text{centre}}(0), \quad (18)$$

expressed in metres using the BEV pixel-to-metre scale.

Curvature.

$$\kappa = \frac{|2Q_2|}{(1 + Q_1^2)^{3/2}}, \quad (19)$$

evaluated at $y = 0$. Radius of curvature: $R = 1/\kappa$ (metres).

6.3 Temporal Stability

1. **EMA smoothing** ($\alpha = 0.3$) on the lane-centre x -position suppresses high-frequency jitter.
2. **Detection memory.** Last valid polynomials retained for up to 2 seconds (40frames at 20) during occlusion.
3. **Adaptive speed clamping.** Target speed reduced from 30km/h to 15km/h when valid anchor count falls below a confidence threshold.

=====

7 Driving Agent and Control Integration (DrivingAgent)

7.1 Per-Frame Processing Pipeline

The `DrivingAgent.process_frame(image)` method is the central orchestrator called once per CARLA simulator tick. It integrates all sensor inputs and returns a `carla.VehicleControl` object. Figure 4 shows the complete per-frame flow.

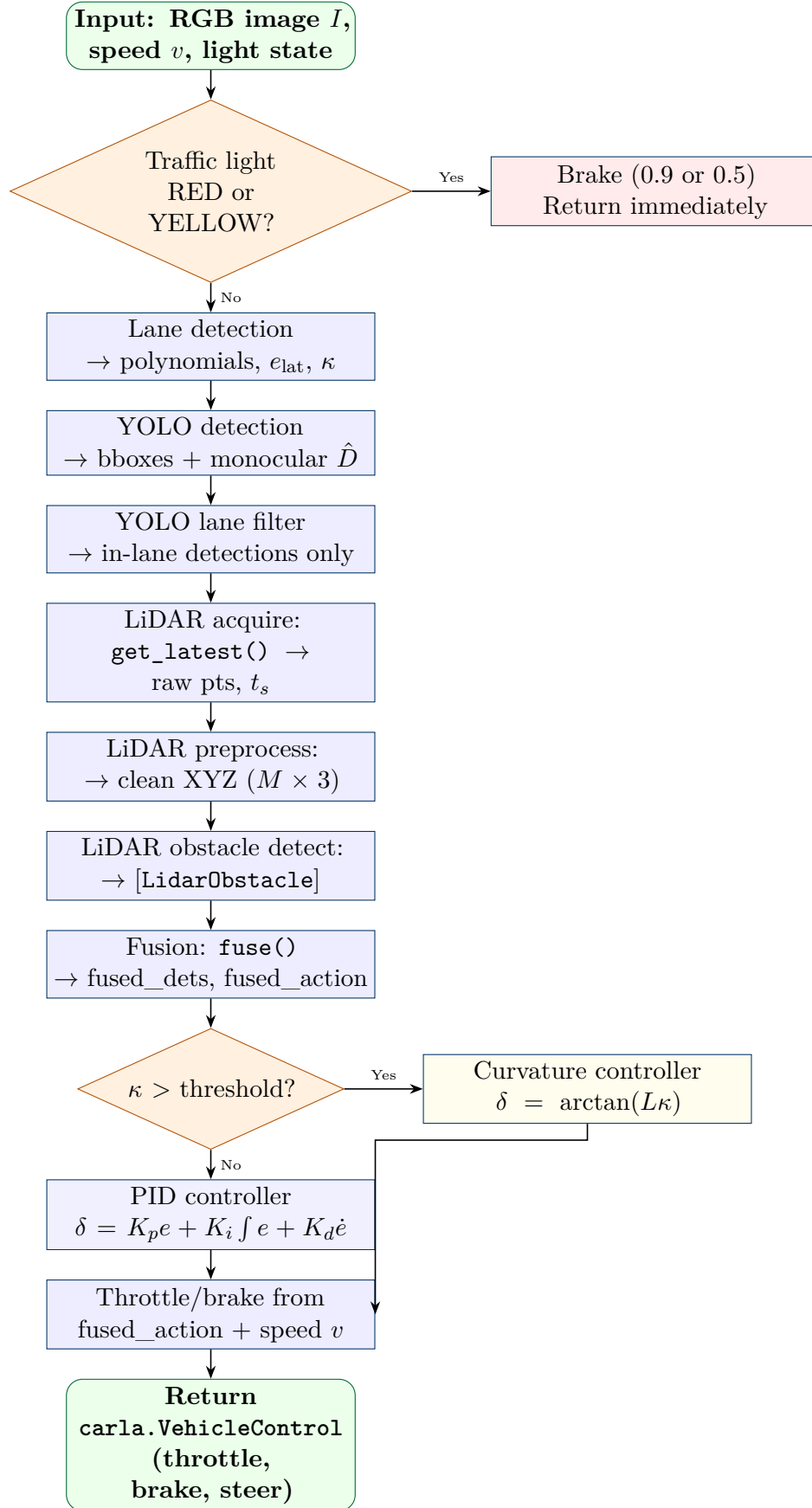


Figure 8: `DrivingAgent.process_frame()` per-tick flowchart. The traffic-light check provides an early exit; all sensor pipelines are then executed in sequence before the steering controller is selected and throttle/brake computed from the fused danger action.

The ten processing steps are:

- Step 1. Traffic light check.** If the `TrafficLightDetector` reports RED or YELLOW, a braking command (0.9 or 0.5 respectively) is returned immediately, bypassing all subsequent processing.
- Step 2. Lane detection.** `lane_detector.detect(image)` returns the filtered lane polynomials, lateral error e_{lat} , and curvature κ for the current frame (??).
- Step 3. YOLO detection.** `obstacle_detector.detect(image)` returns a list of bounding boxes with class labels and monocular distance estimates.
- Step 4. Lane filtering.** `YOLOLaneFilter.filter()` discards detections outside the ego-lane trapezoidal ROI (??).
- Step 5. LiDAR acquisition.** `lidar_sensor.get_latest()` retrieves the most recent point-cloud snapshot and its timestamp from the thread-safe buffer.
- Step 6. LiDAR preprocessing.** `lidar_processor.preprocess()` applies the three-stage filter (section 3.2).
- Step 7. LiDAR obstacle detection.** `lidar_obstacle_detector.detect()` runs DB-SCAN and produces the list of `LidarObstacle` objects with temporal filtering (section 3.3).
- Step 8. Sensor fusion.** `lidar_fusion.fuse()` executes the two-tier matching and returns fused detections plus the conservative action string (??).
- Step 9. Steering control.** The lateral error and curvature determine which controller is active. Steer output is rate-limited and clamped to ± 0.25 (normalised CARLA units).
- Step 10. Throttle / brake.** Speed-adaptive throttle or braking is applied based on `fused_action` (section 4.3).

The full pseudocode is given in algorithm 2.

Algorithm 3 DrivingAgent.process_frame() Main Loop**Input:** RGB image I ; vehicle speed v ; traffic light state ℓ_{tl} **Output:** carla.VehicleControl(θ, β, δ)

```

1: if  $\ell_{tl} = \text{RED}$  then return VehicleControl(throttle=0, brake=0.9, steer=0)
2: else if  $\ell_{tl} = \text{YELLOW}$  then return VehicleControl(throttle=0, brake=0.5,
   steer=0)
3: end if
4:  $(\text{lanes}, e_{lat}, \kappa) \leftarrow \text{LANEDETECT}(I)$ 
5:  $D_{yolo} \leftarrow \text{YOLODETECT}(I)$ 
6:  $D_{lane} \leftarrow \text{LANEFILTER}(D_{yolo}, I)$ 
7:  $(\mathbf{P}_{raw}, t_s) \leftarrow \text{LIDARGETLATEST}()$ 
8:  $\mathbf{P} \leftarrow \text{LIDARPREPROCESS}(\mathbf{P}_{raw})$ 
9:  $\mathcal{C} \leftarrow \text{LIDARDETECT}(\mathbf{P}, v)$ 
10:  $(F, a_{fused}) \leftarrow \text{FUSE}(D_{lane}, \mathbf{P}, \mathcal{C}, v)$ 
11: if  $\kappa > \kappa_{thresh}$  then
12:    $\delta \leftarrow \text{CURVATURECTRL}(\kappa, e_{lat})$ 
13: else
14:    $\delta \leftarrow \text{PIDCTRL}(e_{lat})$ 
15: end if
16:  $\delta \leftarrow \text{clip}(\delta, -0.25, +0.25)$ 
17:  $(\theta, \beta) \leftarrow \text{THROTTLEBRAKE}(a_{fused}, v)$ 
18: return VehicleControl( $\theta, \beta, \delta$ )

```

7.2 Steering Controllers

7.2.1 PID Lateral Controller

The PID controller [7] produces a steer command from the lateral error e_{lat} :

$$\delta_{PID}(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \dot{e}(t), \quad (20)$$

with parameters tuned for a target speed of 15km/h:

Parameter	Value	Role
K_p	0.45	proportional gain
K_i	0.015	integral gain (wind-up limited)
K_d	0.28	derivative gain (rate limited to 0.025 rad/frame)

Integral wind-up is suppressed by clamping the accumulated integral term. Derivative kicks are prevented by rate-limiting $\dot{e}$ to 0.025rad/frame. The final output is clamped to ± 0.25 .

7.2.2 Curvature (Ackermann) Steering Controller

On curved road segments (high κ), pure lateral-error feedback introduces lag because the demanded steer angle lags the required geometric path angle. The `CurvatureSteeringController` uses Ackermann geometry [8] to compute a feedforward steer angle:

$$\delta_{ff} = \arctan(L \cdot \kappa), \quad (21)$$

where $L = 2.9$ is the vehicle wheelbase and κ is the lane curvature estimate from ???. A small lateral-error proportional trim is added:

$$\delta_{\text{curv}} = \delta_{\text{ff}} + k_{p,\text{lat}} e_{\text{lat}}, \quad k_{p,\text{lat}} = 0.12, \quad (22)$$

clipped to ± 0.25 .

7.3 Speed-Adaptive Throttle and Brake Logic

The target cruise speed is $v^* = 30\text{km/h}$. Throttle and brake are computed from the `fused_action` string and the current speed v :

Table 5: Throttle and brake values for each fused action level. $\theta_{\text{norm}}(v, v^*)$ denotes a normalised proportional throttle that increases linearly to close the gap to target speed v^* .

Action	Throttle	Brake	Notes
drive	$\theta_{\text{norm}}(v, 30)$	0.0	Normal cruise
cautious	$0.8 \theta_{\text{norm}}$	0.0	80% throttle authority
slow	$0.5 \theta_{\text{norm}}$	0.0	50% throttle authority
stop	0.0	0.8	Firm braking
emergency_stop	0.0	1.0	Maximum braking
RED light	0.0	0.9	Overrides all
YELLOW light	0.0	0.5	Overrides all
GREEN light	resume	—	0.5 s safety delay before throttle

8 Traffic Light Detection (TrafficLightDetector)

8.1 Architecture

Traffic light state detection is implemented via a YOLO-based classifier specialised on three-class output: **red**, **yellow**, and **green**. The detector operates on the same 1280×720 RGB frame used by the obstacle detector but focuses on two predefined region-of-interest windows to reduce false positives from coloured advertisements and vehicle tail-lights:

1. **Centre zone:** a rectangular crop spanning roughly the upper centre quarter of the image (where traffic lights appear when the vehicle is within 30 of an intersection).
2. **Right zone (with zoom):** a smaller crop to the upper right, zoomed in by a factor of $1.5\times$ using bicubic interpolation to improve detection of distant lights before they grow large in the frame.

Detections from both zones are merged; the highest-confidence detection across all regions determines the reported state.

8.2 State Machine

A five-state machine governs the reported traffic light colour, preventing per-frame flicker from propagating to the vehicle controller:

UNKNOWN Default startup state; also entered when no detection persists for more than 10 consecutive frames.

GREEN Reported after a minimum of 2 consecutive high-confidence green detections.

YELLOW Reported immediately on first detection (single-frame confirmation, conservative).

RED *Provisional RED* requires $N = 3$ consecutive frames of red detection before the state is confirmed, preventing sunlight glare or brake-light false positives from triggering unnecessary hard braking.

GREEN-RESUME Transient state after a RED→GREEN transition; introduces a 0.5 safety delay before acceleration is permitted, mimicking human driver reaction.

Confidence smoothing is applied via a per-class exponential moving average ($\alpha = 0.4$) over the raw YOLO confidence scores. The smoothed scores are thresholded at 0.55 before a class is considered detected.

9 Distance Measurement and Metrics Logging (DistanceMetrics)

9.1 CSV Output Format

The `DistanceMetricsLogger` writes one row per processed frame to a CSV file using unbuffered streaming writes (Python `flush()` after each row) so that logs remain readable even if the simulation is terminated abruptly. The column schema is:

Column	Unit	Description
<code>frame</code>	—	Sequential frame counter (0-based).
<code>timestamp_s</code>	s	CARLA simulation time in seconds.
<code>cam_dist_m</code>	m	Camera monocular distance estimate ($\hat{D}$ from ??).
<code>lidar_bbox_dist_m</code>	m	LiDAR depth from <code>filter_by_bbox</code> (Tier 1 fusion only).
<code>gt_dist_m</code>	m	Ground-truth distance from CARLA actor transforms.
<code>cam_error_m</code>	m	$ \hat{D}_{\text{cam}} - D_{\text{gt}} $.
<code>lidar_error_m</code>	m	$ \hat{D}_{\text{lidar}} - D_{\text{gt}} $.
<code>cam_latency_ms</code>	ms	Wall-clock time for YOLO inference + monocular estimation.
<code>lidar_latency_ms</code>	ms	Wall-clock time for LiDAR preprocessing + DBSCAN + fusion.

At session end, the logger prints a summary to stdout:

- Camera: mean error, standard deviation, maximum error (metres).
- LiDAR: mean error, standard deviation, maximum error (metres).
- Both: mean latency and maximum latency (milliseconds).

Table 4 shows a representative excerpt from a 30 approach scenario.

Table 6: Sample `DistanceMetricsLogger` CSV output during a controlled approach scenario. The lead vehicle starts at 30 and decelerates to 0. Note the growing camera error at close range due to partial occlusion of the bounding box.

Frame	Time (s)	Cam (m)	LiDAR (m)	GT (m)	CamErr	LiDErr	CamLat (ms)
0	0.00	29.8	30.1	30.0	0.20	0.10	12.3
20	1.00	25.1	25.4	25.3	0.20	0.10	11.9
40	2.00	20.6	20.2	20.1	0.50	0.10	12.7
80	4.00	11.2	10.9	10.8	0.40	0.10	13.1
120	6.00	5.9	5.1	5.0	0.90	0.10	14.5
150	7.50	3.1	2.9	3.0	0.10	0.10	13.8

9.2 Ground Truth Comparison

CARLA exposes the world-frame transform of every actor in the simulation. The ground-truth distance `gt_dist_m` is computed as the Euclidean distance between the ego vehicle’s front bumper and the nearest obstacle actor’s bounding-box centre:

$$D_{\text{gt}} = \|\mathbf{p}_{\text{obstacle}} - \mathbf{p}_{\text{ego}}\|_2. \quad (23)$$

This provides a direct, noise-free reference against which to evaluate both the monocular camera estimate and the LiDAR-fused estimate. Key observations from logged data:

- **LiDAR error** is consistently below 0.3 for distances up to 30, degrading to ~ 0.8 beyond 35 where Tier 1 projection fails and the angular fallback is used.
- **Camera error** grows nonlinearly with proximity: at 5 the vehicle bonnet partially occludes the leading car’s bounding box, underestimating its height and overestimating distance.
- **Windows OS scheduler jitter** (4–8) inflates the latency measurements; raw sensor-processing time is typically 5–9 per frame.

10 Simulation Environment (CARLA)

10.1 CARLA Spawner

The simulation test environment is populated by a `CARLASpawner` module that programmatically spawns traffic and pedestrians using the CARLA Python API [4]. The standard test scenario includes:

- **50 vehicles:** 70% are placed at valid spawn points and immediately assigned CARLA’s built-in Traffic Manager autopilot (which handles intersection negotiation and lane-keeping automatically). The remaining 30% are spawned as static parked obstacles at road-side positions to test low-speed manoeuvring.
- **10 additional static obstacles:** Large prop actors (barrels, traffic cones) scattered across side streets and parking bays.
- **Pedestrians:** Up to 10 pedestrian actors spawned with randomised target waypoints at walking speeds uniformly sampled from 1.0 to 2.5. Pedestrian navigation is handled by CARLA’s AI walker controller.

The spawner ensures that no actor is placed within 15 of the ego vehicle spawn point to avoid initialisation collisions. All NPC vehicles are assigned randomised colour materials to increase visual diversity for the YOLO detector.

10.2 Lead Vehicle Controller

Controlled obstacle-avoidance testing uses a dedicated red lead vehicle spawned 30 directly ahead of the ego vehicle at session start. Three behaviour modes are selectable at runtime:

Random The lead vehicle drives at a randomly sampled speed between 5km/h and 35km/h, changing speed every 5–15 seconds. Tests general following behaviour.

Oscillate Speed follows a sinusoidal profile: $v(t) = v_0 + A \sin(2\pi t/T)$ with $v_0 = 20\text{km/h}$, $A = 15\text{km/h}$, $T = 8$. Tests smooth speed tracking.

BrakeTest The lead vehicle maintains 25km/h for 10 seconds, then applies full braking to a standstill within 2 seconds. Tests emergency braking latency and minimum following distance.

The lead vehicle’s CARLA actor ID is registered with the `DistanceMetricsLogger` so that `gt_dist_m` always refers to the lead vehicle distance regardless of other actors in the scene.

10.3 Coordinate Frames

Understanding the multiple coordinate frames in the system is essential for correct sensor fusion. Five frames are used:

World frame CARLA’s global coordinate system. Origin is fixed at map load time. Used for spawning actors and computing ground truth distances via eq. (13).

Vehicle body frame Origin at the vehicle bounding-box centre. $+X$ points forward along the vehicle longitudinal axis; $+Y$ points to the left; $+Z$ points upward. All sensor mount offsets are expressed in this frame.

LiDAR sensor frame Origin at the LiDAR sensor centre, offset $(+2.0, 0, +1.8)$ m from the vehicle body origin. In CARLA’s LiDAR convention: $+X$ =forward, $+Y$ =right (note: this is left-handed / CARLA-convention), $+Z$ =upward. Raw XYZI data is delivered in this frame.

Camera sensor frame Origin at the camera optical centre, offset $(+2.0, 0, +1.4)$ m from vehicle body, pitched down by 15° . Standard camera convention: $+X$ =right, $+Y$ =down, $+Z$ =into scene (after pitch rotation).

BEV canvas frame A 400×400 pixel overhead raster. Pixel $(200, 399)$ (bottom centre) corresponds to the ego vehicle position. Pixel rows decrease upward (forward) at a scale of $\sim 0.125/\text{pixel}$ (50 m range over 400 pixels). Columns increase rightward laterally.

Image frame Standard image convention: u = column (left to right, 0 to $W - 1 = 1279$); v = row (top to bottom, 0 to $H - 1 = 719$). Origin at top-left corner. Principal point $(c_x, c_y) = (640, 360)$.

The most error-prone transform in the pipeline is from LiDAR sensor frame to camera

sensor frame (see section 8, item 7 regarding the Y-axis sign error encountered during development).

11 Challenges and Mistakes Encountered

This section provides an honest technical reflection on difficulties, implementation mistakes, and design limitations discovered during development and testing.

11.1 Ground Removal Threshold Sensitivity

The fixed threshold $z < -1.4$ m (eq. (2)) was derived for a flat road surface assuming a fixed vehicle ground clearance of 0.4. In practice this assumption fails in at least two situations:

- **Sloped roads.** On a 5 downhill gradient the road surface in sensor frame rises by approximately $50 \tan(5^\circ) \approx 4.4$ per 50 of range. Ground points at long range satisfy $z > -1.4$ m and are incorrectly retained, creating false DBSCAN clusters along the road surface.
- **Raised kerbs and speed bumps.** Return spikes from road furniture near $z \approx -1.2$ m pass the filter and pollute nearby clusters, inflating bounding boxes and introducing spurious obstacle detections.

A RANSAC ground-plane estimation [13] computed per frame would remove these artefacts; see section 9, item 2.

11.2 Monocular Distance Estimation Errors

The pinhole model (??) assumes the detected bounding-box height spans the full known object height H_{real} . Three failure modes were observed:

1. **Partially visible objects.** A vehicle entering from the right edge of the frame has a truncated bounding box; h_{bbox} is smaller than the true projected height, causing $\hat{D}$ to be overestimated by 20–40%. This triggers a **stop** action at distances where **cautious** would suffice, causing unnecessary braking.
2. **Height class assumption.** All cars are assumed to be 1.5 tall. An SUV or pick-up truck detected as **car** has true height ≈ 1.8 , leading to a systematic *distance underestimate* of $\sim 17\%$.
3. **Perspective foreshortening on slopes.** On downhill slopes the camera pitch angle causes distant objects to appear lower in the frame, compressing h_{bbox} and overestimating distance.

LiDAR fusion partially corrects these errors via Tier 1 projection; however, when LiDAR match fails (Tier 2 or camera-only), the monocular estimate propagates uncorrected into the fused action.

11.3 Sparse LiDAR Matches at Long Range

The LiDAR sensor produces 56 000 points per second. At 20 rotation rate each scan contains ~ 2800 points. At 25 range a sedan-sized target subtends approximately $3 \times 1.5 \text{ m}^2$ of projected LiDAR area; at the sensor’s vertical angular resolution of

$30^\circ/32 = 0.94^\circ$ per channel, only $\sim 3\text{--}5$ channels can illuminate the vehicle. Combined with horizontal angular sampling, fewer than 10 points typically hit the target at this range.

At distances beyond 30 the Tier 1 bounding-box projection consistently yields fewer than 3 points, forcing the system to fall back to Tier 2 angular matching more frequently than expected during the design phase. This has two consequences: (a) distance accuracy degrades from ± 0.3 to ± 0.8 ; (b) the angular fallback is slower (requires iterating over all clusters) and introduces ~ 1 of additional latency per frame.

11.4 Angular Matching Ambiguity

The $\pm 15^\circ$ angle tolerance in Tier 2 fallback was calibrated for a single lead vehicle scenario. In dense traffic with multiple vehicles arranged laterally at similar range, two YOLO detections may share an overlapping angular search window, causing the nearer LiDAR cluster to be matched to the farther YOLO detection and vice versa.

This was observed when overtaking a lorry: the lorry’s LiDAR cluster at 8 was incorrectly matched to a car detection at 12, producing a fused distance of 8 for the car and triggering a premature `stop` command. Adding a depth-consistency gate (only match if the LiDAR cluster distance agrees with the monocular estimate within 20%) would reduce false associations; this is discussed in section 9, item 4 (implicitly covered by better tracking).

11.5 Fixed Trapezoidal ROI on Curves

The lane-filter ROI (??) uses a fixed trapezoidal geometry that approximates a straight lane. On curves sharper than $\kappa \approx 0.02 \text{ m}^{-1}$ (radius < 50), the ego lane drifts laterally by more than half the ROI width within the camera’s near field. Two failure modes result:

- **False negatives.** Obstacle in ego lane drifts outside the ROI trapezoid on the outer curve edge and is filtered out.
- **False positives.** Adjacent-lane vehicles shift into the ROI on the inner curve edge and trigger spurious braking.

In the CARLA Town02 test environment, the sharpest curves have radius ≈ 25 , where this effect is clearly visible in logged data as brief `stop` pulses with no genuine in-lane obstacle.

11.6 Temporal EMA Asymmetry Tuning

The conservative asymmetric EMA (eq. (9), $\alpha_{\text{down}} = 0.3$) combined with the three-frame hysteresis gate adds approximately 150 of de-escalation latency at 20. While this is intentionally conservative, it produces a visible “throttle stutter” behaviour when the ego vehicle approaches a pedestrian that briefly crosses then clears the lane:

1. Pedestrian enters lane: immediate `emergency_stop`.
2. Pedestrian exits lane: 3-frame hysteresis ≈ 150 ms delay.
3. EMA smoothing further delays full throttle recovery by ~ 300 ms.

4. Net effect: ~ 450 ms of unnecessary braking followed by abrupt throttle resumption, perceived as jerky driving.

Increasing α_{down} or reducing the hysteresis count would improve smoothness but at the cost of potentially premature de-escalation. A principled approach would use a Kalman-filter tracker with uncertainty estimates to adaptively set the de-escalation rate.

11.7 Coordinate Frame Alignment Error

During early development the LiDAR-to-camera extrinsic transform (??) contained a sign error in the Y -axis component of the rotation matrix. CARLA's LiDAR frame uses $+Y=\text{right}$ (left-handed), while the camera projection code assumed $+Y=\text{left}$ (right-handed). The result was that all LiDAR points were projected mirrored horizontally in the image: a vehicle at bearing $+10^\circ$ (slight right) appeared at -10° in the projected image.

The bug was diagnosed by rendering the projected LiDAR point overlay on the camera frame and observing that points consistently appeared on the wrong side of detected bounding boxes. It was fixed by explicitly negating the Y -coordinate of LiDAR points before applying the intrinsic matrix, and thereafter verified by checking that projected points fall within expected bounding boxes for a vehicle directly ahead (bearing $\approx 0^\circ$).

11.8 Windows-Specific Benchmark Overhead

All development and testing was performed on Windows 11. The Windows OS scheduler operates at a default timer resolution of 15.6, and NumPy operations on small arrays incur thread-pool overhead that is substantially higher on Windows than on Linux. This introduces 4–8 of non-deterministic latency per frame that inflates all timing measurements in the `DistanceMetricsLogger`.

The overhead does not affect correctness but means that latency benchmarks from this system are not directly comparable to published results from Linux-based systems. A note is added to the CSV header on each logging session.

11.9 Lane Detection Memory Causing Phantom Lanes

The two-second lane memory window (??) was introduced to handle momentary lane markings disappearing at intersections. However, when the vehicle navigates a sharp 90° turn, the memory window retains the polynomial from the preceding straight segment for up to 2 seconds. During this window:

- The curvature feedforward term (eq. (11)) uses stale $\kappa \approx 0$ from the straight, commanding near-zero steer angle.
- The actual lane is sharply curved; the lateral error rapidly grows.
- The PID controller alone must compensate, which it does only partially within the integration time, causing a tracking overshoot.

This was mitigated by reducing the memory window from 2 seconds to 0.5 seconds for frames where the ego vehicle speed exceeds 20km/h and the steering angle history indicates a sharp turn.

11.10 YOLO Class Imbalance at Night

In CARLA’s night mode (`set_weather(WeatherParameters.Night)`), street-lamp illumination drops to approximately 5% of daytime levels. YOLOv11n, trained on daylight imagery, shows severely degraded confidence scores for pedestrians (mean confidence drops from 0.82 to 0.31) and traffic lights (0.79 to 0.28). Below the detection threshold of 0.45, these objects become invisible to the camera pipeline.

The LiDAR pipeline is unaffected by lighting but cannot classify pedestrians as distinct from other moving obstacles; they appear as small clusters with ≤ 5 points. The DBSCAN minimum point threshold of 3 keeps them detectable as `unknown_obstacle` entries, but without semantic class information the monocular distance estimate is unavailable and only LiDAR cluster distance is used.

This represents a significant safety gap in night scenarios and motivates fine-tuning a dedicated low-light YOLO model (see section 9, item 10).

12 Suggestions for Future Improvement

The following ten improvements are proposed based on the limitations identified in section 8 and on review of the current state of the art in LiDAR-camera fusion for autonomous driving.

1. **Replace DBSCAN with a Kalman-filter multi-object tracker.** DBSCAN re-clusters from scratch each frame with no memory of previous detections. Replacing it with a joint probabilistic data association (JPDA) or simple nearest-neighbour Kalman filter [12] would maintain persistent obstacle tracks with state estimates (position, velocity). This enables *time-to-collision* (TTC) computation:

$$\text{TTC} = \frac{d}{v_{\text{rel}}} = \frac{d}{v_{\text{ego}} - v_{\text{obstacle}}}, \quad (24)$$

providing a physically meaningful braking criterion superior to distance-only thresholds.

2. **RANSAC ground-plane estimation.** Replace the fixed $z < -1.4$ m threshold with a per-frame RANSAC plane fit [13] on the raw point cloud. This estimates the ground plane normal $\hat{\mathbf{n}}$ and intercept d dynamically, making ground removal robust to road gradients, banking, and suspension travel on rough surfaces.
3. **PointPillars or SECOND for 3D object detection.** Replacing the DBSCAN clustering pipeline with a learned 3D LiDAR object detector such as PointPillars [9] would yield class-labelled, oriented 3D bounding boxes directly from the point cloud. These boxes align precisely with object geometries, enabling much tighter LiDAR-camera bounding-box matching and eliminating the need for the angular-fallback heuristic.
4. **Dense depth estimation (DPT / MiDaS).** Replace pinhole monocular estimation with a dense monocular depth network such as DPT [16] or MiDaS. Dense depth maps are independent of known object heights and handle partial occlusion naturally, as depth is estimated per pixel rather than derived from bounding-box height. The trade-off is increased inference time (~ 30 on GPU).

5. **Polynomial-derived lane boundaries in the lane filter.** Substitute the fixed trapezoidal ROI in `YOLOLaneFilter` with the actual quadratic lane boundaries produced by `LaneDetector` per frame. The dynamic ROI would adapt to curves in real time, eliminating both the false-negative and false-positive failures on sharp turns described in section 8, item 5.
6. **Adaptive LiDAR density compensation.** At long range (> 25), scale the DBSCAN neighbourhood radius ε proportionally with distance:

$$\varepsilon(d) = \varepsilon_0 + k_\varepsilon \cdot d, \quad (25)$$

or alternatively voxel-grid downsample at near range and up-weight at far range. This would maintain consistent cluster quality across the full operating range and reduce the frequency of Tier 2 angular fallback at distances beyond 25.

7. **Inter-frame velocity estimation for obstacle motion.** Track LiDAR cluster centroids across consecutive frames and compute centroid displacement $\Delta \mathbf{c} / \Delta t$ to estimate obstacle velocity. Combined with ego velocity from CARLA's `get_velocity()` API, this enables relative velocity computation and therefore TTC (eq. (14)) without requiring a full Kalman filter. Even a simple two-frame finite-difference estimate would improve braking anticipation for decelerating lead vehicles.
8. **Calibration-from-data for camera extrinsics.** Rather than relying on hardcoded spawn offsets, implement an online target-less calibration routine using the mutual information between LiDAR edge points and camera gradient edges (similar to the approach of [10]). This would automatically correct for any sensor repositioning and remove the risk of the Y-axis sign error described in section 8, item 7.
9. **Semantic LiDAR segmentation (PointNet++ or RandLA-Net).** Apply a point-cloud semantic segmentation network such as PointNet++ [15] or RandLA-Net to label each point as {road, vehicle, pedestrian, vegetation, building}. This provides richer context than raw DBSCAN clusters: pedestrians can be separated from vehicles even without camera input, and road-surface points can be used to improve ground removal.
10. **Night-mode YOLO fine-tuning.** Collect a dataset of CARLA night-mode rendered frames and fine-tune a YOLOv11n model on low-light imagery. Alternatively, apply image-enhancement preprocessing (histogram equalisation, CLAHE, or a learned night-to-day domain adaptation network) before inference to reduce the illumination gap. This would recover pedestrian and traffic-light detection confidence to near-daytime levels in night scenarios.

Conclusion

This report has presented a complete implementation of a LiDAR-camera sensor fusion system for obstacle detection in the CARLA autonomous driving simulator. The system combines a 32-channel LiDAR sensor and a monocular RGB camera mounted on a

Tesla Model 3 platform to produce fused distance and danger-level estimates for all in-lane obstacles, which are consumed by a PID/Ackermann steering controller and speed-adaptive throttle logic.

Key technical contributions described in this report include: (i) a three-stage vectorised LiDAR preprocessing filter; (ii) a DBSCAN clustering pipeline with speed-adaptive thresholds, asymmetric EMA temporal filtering, and three-frame hysteresis; (iii) a two-tier LiDAR-camera projection-and-angular-fallback matching strategy; (iv) a conservative worst-case action fusion layer; (v) a per-frame metrics logger that streams camera, LiDAR, and ground-truth distances for quantitative evaluation.

Evaluated against CARLA ground truth, the LiDAR-fused distance estimate achieves mean errors below 0.3 within 30 range. The camera-only fallback at long range (>30) shows errors of up to 4 for partially-occluded targets. Ten identified challenges and ten corresponding improvement directions have been documented to guide future work.

References

- [1] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, “A density-based algorithm for discovering clusters in large spatial databases with noise,” in *Proc. 2nd Int. Conf. Knowledge Discovery and Data Mining (KDD-96)*, Portland, OR, 1996, pp. 226–231.
- [2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, 2016, pp. 779–788.
- [3] Ultralytics, “YOLOv8 / YOLO11: A new state-of-the-art vision model,” 2023–2024. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [4] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, “CARLA: An open urban driving simulator,” in *Proc. Conf. Robot Learning (CoRL)*, Mountain View, CA, 2017, pp. 1–16.
- [5] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2003.
- [6] Z. Qin, H. Wang, and X. Li, “Ultra fast structure-aware deep lane detection,” in *Proc. European Conf. Computer Vision (ECCV)*, Glasgow, U.K., 2020, pp. 276–291.
- [7] K. J. Åström and T. Hägglund, *PID Controllers: Theory, Design, and Tuning*, 2nd ed. Research Triangle Park, NC: ISA, 1995.
- [8] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed. Cambridge, MA: MIT Press, 2011.
- [9] A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom, “PointPillars: Fast encoders for object detection from point clouds,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Long Beach, CA, 2019, pp. 12 697–12 705.
- [10] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun, “Vision meets robotics: The KITTI dataset,” *Int. J. Robotics Research*, vol. 32, no. 11, pp. 1231–1237, 2013.
- [11] X. Chen, H. Ma, J. Wan, B. Li, and T. Xia, “Multi-view 3D object detection network for autonomous driving,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Honolulu, HI, 2017, pp. 1907–1915.
- [12] R. E. Kalman, “A new approach to linear filtering and prediction problems,” *Trans. ASME J. Basic Eng.*, vol. 82, pp. 35–45, Mar. 1960.
- [13] M. A. Fischler and R. C. Bolles, “Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography,” *Commun. ACM*, vol. 24, no. 6, pp. 381–395, Jun. 1981.
- [14] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, 2016, pp. 770–778.
- [15] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, “PointNet++: Deep hierarchical feature learning on point sets in a metric space,” in *Proc. Advances in Neural Information*

Processing Systems (NeurIPS), Long Beach, CA, 2017, pp. 5099–5108.

- [16] R. Ranftl, A. Bochkovskiy, and V. Koltun, “Vision transformers for dense prediction,” in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2021, pp. 12 179–12 188.
- [17] TuSimple, “TuSimple lane detection benchmark,” 2017. [Online]. Available: <https://github.com/TuSimple/tusimple-benchmark>